



软件系统设计复习

设计原则

- 对于面向对象的软件系统设计来说，在支持可维护性的同时，需要提高系统的可复用性。
- 软件的复用可以提高软件的开发效率，提高软件质量，节约开发成本，恰当的复用还可以改善系统的可维护性。
- 单一职责原则要求在软件系统中，一个类只负责一个功能领域中的相应职责。
- 开闭原则要求一个软件实体应当对扩展开放，对修改关闭，即在不修改源代码的基础上扩展一个系统的行为。
- 里氏代换原则可以通俗表述为在软件中如果能够使用基类对象，那么一定能够使用其子类对象。

设计原则

- 依赖倒转原则要求抽象不应该依赖于细节，细节应该依赖于抽象；要针对接口编程，不要针对实现编程。
- 接口隔离原则要求客户端不应该依赖那些它不需要的接口，即将一些大的接口细化成一些小的接口供客户端使用。
- 合成复用原则要求复用时尽量使用对象组合，而不使用继承。
- 迪米特法则要求一个软件实体应当尽可能少的与其他实体发生相互作用。

设计原则

- 目标：开闭原则
- 指导：最小知识原则
- 基础：单一职责原则、可变性封装原则
- 实现：依赖倒转原则、合成复用原则、里氏代换原则、接口隔离原则

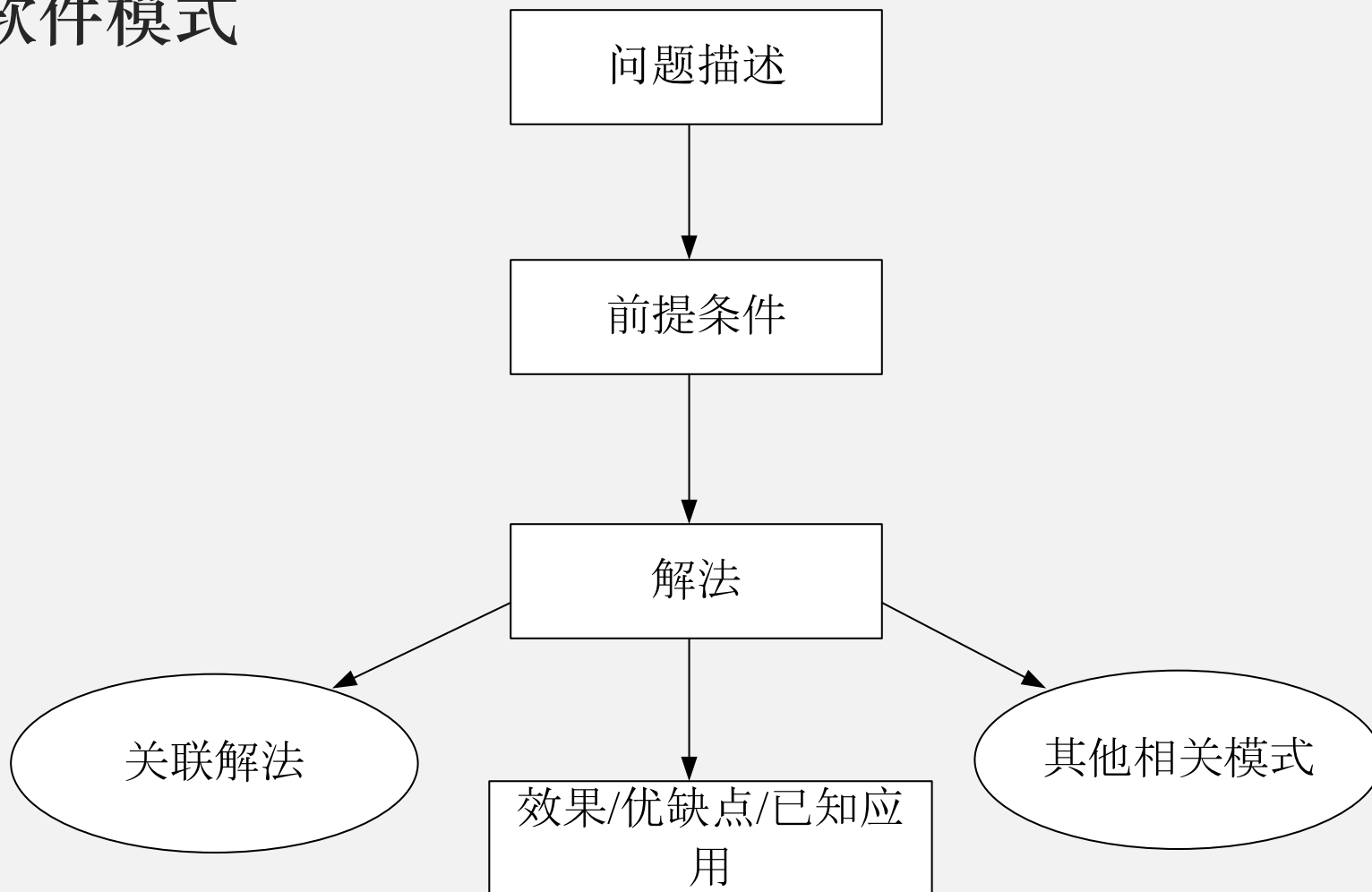
软件模式

- 软件模式

- 软件模式是将模式的一般概念应用于软件开发领域，即**软件开发的总体指导思路或参照样板**。软件模式并非仅限于设计模式，还包括架构模式、分析模式和过程模式等，实际上，**在软件生存期的每一个阶段都存在着一些被认同的模式**。
- 软件模式可以认为是对软件开发这一特定“问题”的“解法”的某种统一表示，**软件模式等于一定条件下的出现的问题以及解法**。软件模式的基础结构由4个部分构成：**问题描述、前提条件（环境或约束条件）、解法和效果**。

软件模式

- 软件模式



软件模式

- 软件模式
 - 软件模式与具体的应用领域无关，在模式发现过程中需要遵循大三律(Rule of Three)，即只有经过三个以上不同类型（或不同领域）的系统的校验，一个解决方案才能从候选模式升格为模式。

设计模式的定义

- 设计模式的定义
 - 设计模式(Design Pattern)是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结，使用设计模式是为了可重用代码、让代码更容易被他人理解、保证代码可靠性。

设计模式的基本要素

- 设计模式的基本要素
 - 设计模式一般有如下几个基本要素：模式名称、问题、目的、解决方案、效果、实例代码和相关设计模式，其中的关键元素包括以下四个方面：
 - 模式名称 (Pattern name)
 - 问题 (Problem)
 - 解决方案 (Solution)
 - 效果 (Consequences)

设计模式的分类

- 设计模式的分类

- 根据其目的（模式是用来做什么的）可分为创建型(Creational)，结构型(Structural)和行为型(Behavioral)三种：
 - 创建型模式主要用于创建对象。
 - 结构型模式主要用于处理类或对象的组合。
 - 行为型模式主要用于描述对类或对象怎样交互和怎样分配职责。

设计模式的分类

• 设计模式的分类

- 根据**范围**，即模式主要是用于处理类之间关系还是处理对象之间的关系，可分为**类模式**和**对象模式**两种：
 - 类模式**处理类和子类之间的关系**，这些关系通过继承建立，在编译时刻就被确定下来，是属于**静态**的。
 - 对象模式**处理对象间的关系**，这些关系在运行时刻变化，更具**动态**性。

设计模式的分类

范围\目的	创建型模式	结构型模式	行为型模式
类模式	工厂方法模式	（类）适配器模式	模板方法模式
对象模式	抽象工厂模式 建造者模式 原型模式 单例模式	（对象）适配器模式 桥接模式 组合模式 装饰模式 外观模式 享元模式 代理模式	命令模式 迭代器模式 中介者模式 观察者模式 状态模式 策略模式

设计模式与类库框架

问：如果设计模式这么好，为什么没人构建一个包含这些模式的库，这样就不用自己去弄了呢？

答：设计模式比库的层次更高。设计模式告诉我们如何构建类和对象来解决特定问题，而根据我们特定的应用去调整这些设计是我们（开发者）的工作。

问：类库和框架不也是设计模式吗？

答：框架和类库并非设计模式；它们提供特定的实现，供我们关联到自己的代码中。不过，有时类库和框架在其实现中会运用设计模式。这很好，因为一旦理解了设计模式，就能更快地理解那些围绕设计模式构建的应用程序编程接口（APIs）。

Patterns of Patterns

Working together

One of the best ways to use patterns is to get them out of the house so they can interact with other patterns. The more you use patterns the more you're going to see them showing up together in your designs. We have a special name for a set of patterns that work together in a design that can be applied over many problems: a compound pattern. That's right, we are now talking about patterns made of patterns!

- Patterns are often used together and combined within the same design solution.
- A compound pattern combines two or more patterns into a solution that solves a recurring or general problem.

Duck reunion

First, we'll create a Quackable interface.

Like we said, we're starting from scratch. This time around, the Ducks are going to implement a Quackable interface. That way we'll know what things in the simulator can quack() – like Mallard Ducks, Redhead Ducks, Duck Calls, and we might even see the Rubber Duck sneak back in.

```
public interface Quackable {  
    public void quack();  
}
```

Quackables only need to do one thing well: Quack!



Continue

Now, some Ducks that implement Quackable

What good is an interface without some classes to implement it?
Time to create some concrete ducks.

```
public class MallardDuck implements Quackable {  
    public void quack() {  
        System.out.println("Quack");  
    }  
}
```

*Your standard
Mallard duck.*

```
public class RedheadDuck implements Quackable {  
    public void quack() {  
        System.out.println("Quack");  
    }  
}
```

*We've got to have some variation
of species if we want this to be an
interesting simulator.*

Add other ducks

```
public class DuckCall implements Quackable {  
    public void quack() {  
        System.out.println("Kwak");  
    }  
}
```

A DuckCall that quacks but doesn't sound quite like the real thing.

```
public class RubberDuck implements Quackable {  
    public void quack() {  
        System.out.println("Squeak");  
    }  
}
```

A RubberDuck that makes a squeak when it quacks.

Simulator

```
public class DuckSimulator {  
    public static void main(String[] args) {  
        DuckSimulator simulator = new DuckSimulator();  
        simulator.simulate();  
    }
```

Here's our main method to get everything going.

We create a simulator and then call its simulate() method.

```
void simulate() {  
    Quackable mallardDuck = new MallardDuck();  
    Quackable redheadDuck = new RedheadDuck();  
    Quackable duckCall = new DuckCall();  
    Quackable rubberDuck = new RubberDuck();
```

We need some ducks, so here we create one of each Quackable...

```
    System.out.println("\nDuck Simulator");
```

```
    simulate(mallardDuck);  
    simulate(redheadDuck);  
    simulate(duckCall);  
    simulate(rubberDuck);
```

... then we simulate each one.

```
}
```

Here we overload the simulate method to simulate just one duck.

```
void simulate(Quackable duck) {  
    duck.quack();  
}
```

```
}
```

Here we let polymorphism do its magic: no matter what kind of Quackable gets passed in, the simulate() method asks it to quack.

a Goose class that has been hanging around the simulator

```
public class Goose {  
    public void honk() {  
        System.out.println("Honk");  
    }  
}
```

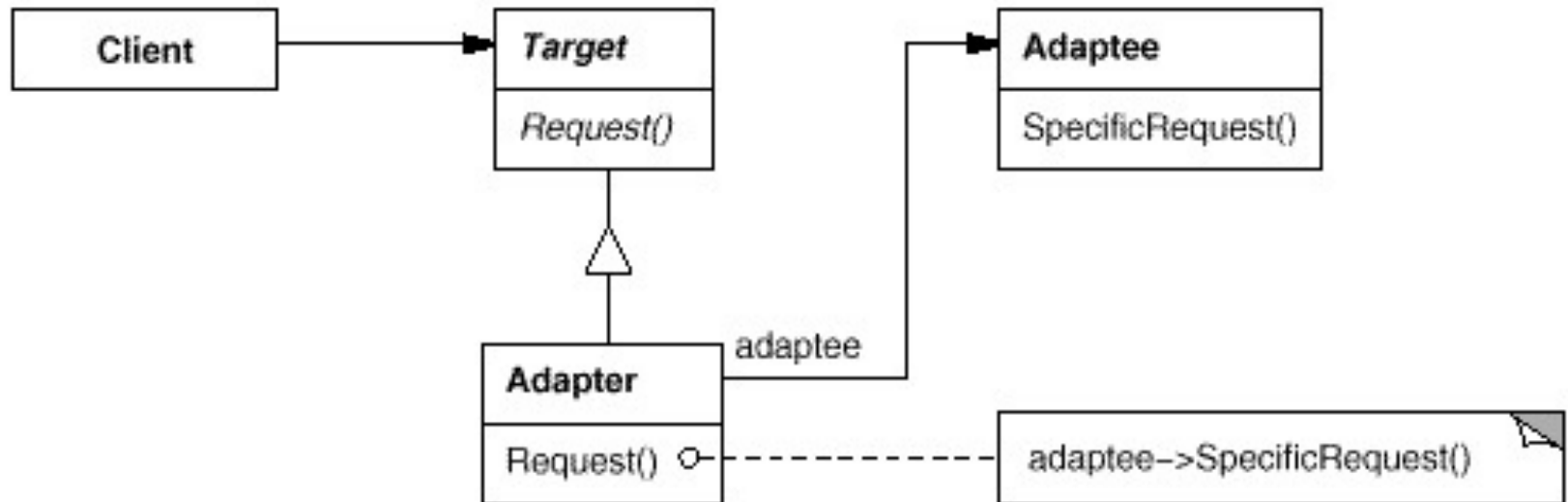


A Goose is a honker,
not a quacker.

Let's say we wanted to be able to use a Goose anywhere we'd want to use a Duck. After all, geese make noise; geese fly; geese swim. Why can't we have Geese in the simulator?
What pattern would allow Geese to easily intermingle with Ducks?

We need a goose adapter

Our simulator expects to see Quackable interfaces. Since geese aren't quackers (they're honkers), we can use an adapter to adapt a goose to a duck.



We need a goose adapter

Our simulator expects to see Quackable interfaces. Since geese aren't quackers (they're honkers), we can use an adapter to adapt a goose to a duck.

```
public class GooseAdapter implements Quackable {  
    Goose goose;  
  
    public GooseAdapter(Goose goose) {  
        this.goose = goose;  
    }  
  
    public void quack() {  
        goose.honk();  
    }  
}
```

Remember, an Adapter implements the target interface, which in this case is Quackable.

← The constructor takes the goose we are going to adapt.

← When quack is called, the call is delegated to the goose's honk() method.

Simulator with goose adapter

```
public class DuckSimulator {  
    public static void main(String[] args) {  
        DuckSimulator simulator = new DuckSimulator();  
        simulator.simulate();  
    }  
    void simulate() {  
        Quackable mallardDuck = new MallardDuck();  
        Quackable redheadDuck = new RedheadDuck();  
        Quackable duckCall = new DuckCall();  
        Quackable rubberDuck = new RubberDuck();  
        Quackable gooseDuck = new GooseAdapter(new Goose());  
  
        System.out.println("\nDuck Simulator: With Goose Adapter");  
  
        simulate(mallardDuck);  
        simulate(redheadDuck);  
        simulate(duckCall);  
        simulate(rubberDuck);  
        simulate(gooseDuck);  
    }  
  
    void simulate(Quackable duck) {  
        duck.quack();  
    }  
}
```

We make a Goose that acts like a Duck by wrapping the Goose in the GooseAdapter.

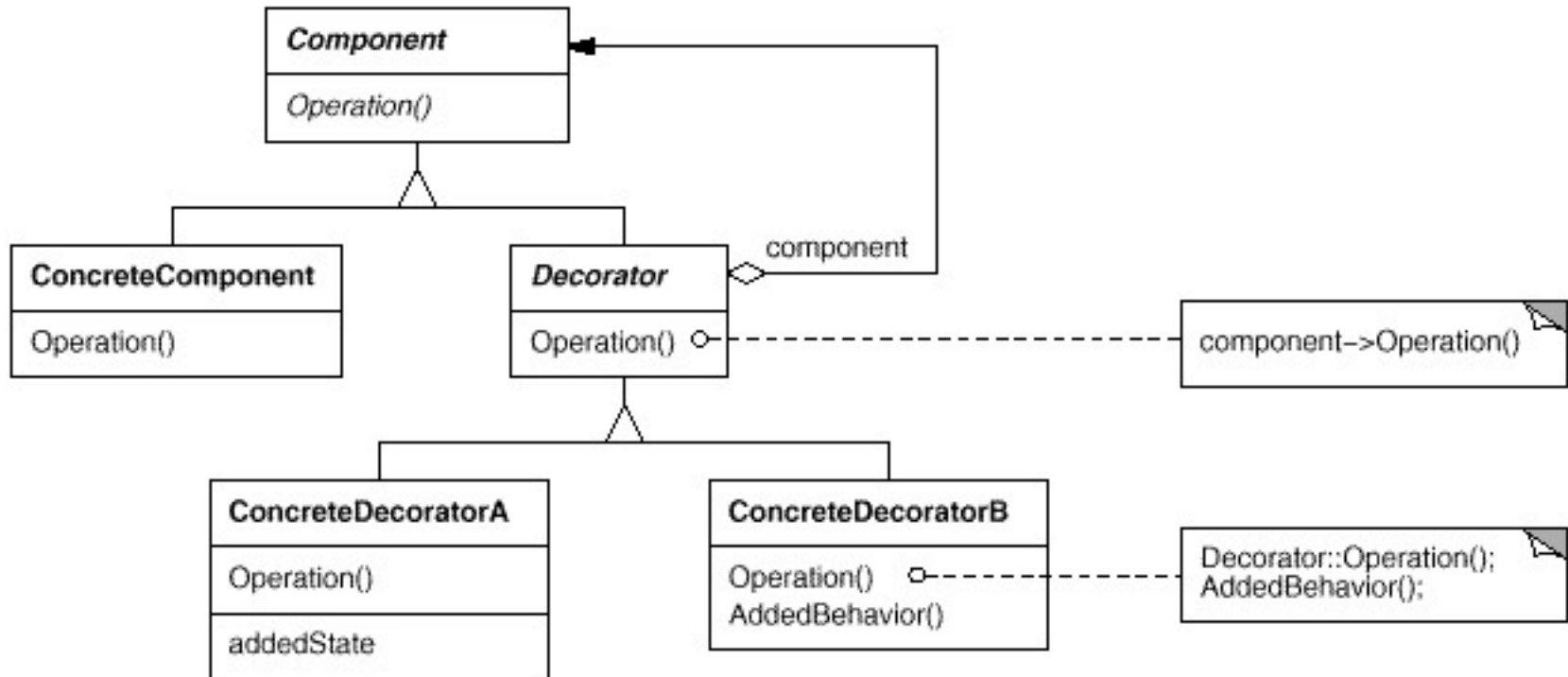
Once the Goose is wrapped, we can treat it just like other duck Quackables.

Quackology

- Quackologists are fascinated by all aspects of Quackable behavior. One thing Quackologists have always wanted to study is the total number of quacks made by a flock of ducks.
- How can we add the ability to count duck quacks without having to change the duck classes?
- Can you think of a pattern that would help?

Quack counts

Let's create a decorator that gives the ducks some new behavior (the behavior of counting) by wrapping them with a decorator object.



Quack counts

Let's create a decorator that gives the ducks some new behavior (the behavior of counting) by wrapping them with a decorator object.

QuackCounter is a decorator

Like with Adapter, we need to implement the target interface.

We've got an instance variable to hold on to the quacker we're decorating.

And we're counting ALL quacks, so we'll use a static variable to keep track.

We get the reference to the Quackable we're decorating in the constructor.

When quack() is called, we delegate the call to the Quackable we're decorating...

... then we increase the number of quacks.

```
public class QuackCounter implements Quackable {
    Quackable duck;
    static int numberOfQuacks;

    public QuackCounter (Quackable duck) {
        this.duck = duck;
    }

    public void quack() {
        duck.quack();
        numberOfQuacks++;
    }

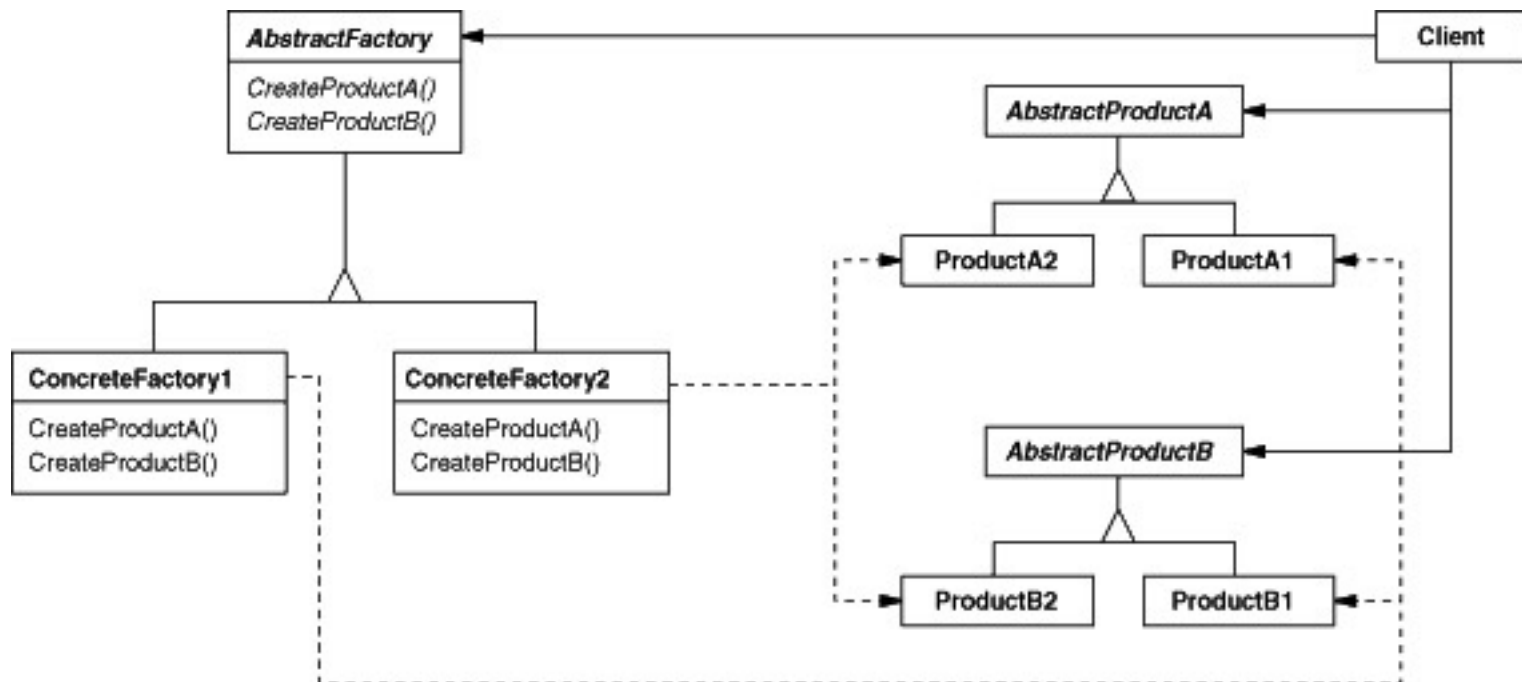
    public static int getQuacks() {
        return numberOfQuacks;
    }
}
```

You have to decorate objects to get decorated behavior

- This quack counting is great. We're learning things we never knew about the little quackers. But we're finding that too many quacks aren't being counted.
- That's the problem with wrapping objects: you have to make sure they get wrapped or they don't get the decorated behavior.
 - Why don't we take the creation of ducks and localize it in one place; in other words, let's take the duck creation and decorating and encapsulate it.
 - What pattern does that sound like?

Factory

Okay, we need some quality control to make sure our ducks get wrapped. We're going to build an entire factory just to produce them. The factory should produce a family of products that consists of different types of ducks, so we're going to use the Abstract Factory Pattern.



Creating a factory that creates ducks without decorators

```
public class DuckFactory extends AbstractDuckFactory {
```

```
    public Quackable createMallardDuck() {  
        return new MallardDuck();  
    }
```

```
    public Quackable createRedheadDuck() {  
        return new RedheadDuck();  
    }
```

```
    public Quackable createDuckCall() {  
        return new DuckCall();  
    }
```

```
    public Quackable createRubberDuck() {  
        return new RubberDuck();  
    }
```

```
}
```



DuckFactory extends the abstract factory.



Each method creates a product: a particular kind of Quackable. The actual product is unknown to the simulator – it just knows it's getting a Quackable.

CountingDuckFactory

CountingDuckFactory
also extends the
abstract factory.



```
public class CountingDuckFactory extends AbstractDuckFactory {  
  
    public Quackable createMallardDuck() {  
        return new QuackCounter(new MallardDuck());  
    }  
  
    public Quackable createRedheadDuck() {  
        return new QuackCounter(new RedheadDuck());  
    }  
  
    public Quackable createDuckCall() {  
        return new QuackCounter(new DuckCall());  
    }  
  
    public Quackable createRubberDuck() {  
        return new QuackCounter(new RubberDuck());  
    }  
}
```

Each method wraps the Quackable with the quack counting decorator. The simulator will never know the difference; it just gets back a Quackable. But now our rangers can be sure that all quacks are being counted.



How Abstract Factory works? We create a polymorphic method that takes a factory and uses it to create objects. By passing in different factories, we get to use different product families in the method.

```
public class DuckSimulator {  
    public static void main(String[] args) {  
        DuckSimulator simulator = new DuckSimulator();  
        AbstractDuckFactory duckFactory = new CountingDuckFactory();  
  
        simulator.simulate(duckFactory);  
    }  
}
```

First we create the factory that we're going to pass into the simulate() method.

```
void simulate(AbstractDuckFactory duckFactory) {  
    Quackable mallardDuck = duckFactory.createMallardDuck();  
    Quackable redheadDuck = duckFactory.createRedheadDuck();  
    Quackable duckCall = duckFactory.createDuckCall();  
    Quackable rubberDuck = duckFactory.createRubberDuck();  
    Quackable gooseDuck = new GooseAdapter(new Goose());  
  
    System.out.println("\nDuck Simulator: With Abstract Factory");  
  
    simulate(mallardDuck);  
    simulate(redheadDuck);  
    simulate(duckCall);  
    simulate(rubberDuck);  
    simulate(gooseDuck);  
  
    System.out.println("The ducks quacked " +  
        QuackCounter.getQuacks() +  
        " times");  
}
```

The simulate() method takes an AbstractDuckFactory and uses it to create ducks rather than instantiating them directly.

```
void simulate(Quackable duck) {  
    duck.quack();  
}
```

Nothing changes here! Same ol' code.

```
}
```

Manage a flock of ducks

- It's getting a little difficult to manage all these different ducks separately. Is there any way you can help us manage ducks as a whole, and perhaps even allow us to manage a few duck "families" that we'd like to keep track of?

This isn't very
manageable!



```
Quackable mallardDuck = duckFactory.createMallardDuck();
Quackable redheadDuck = duckFactory.createRedheadDuck();
Quackable duckCall = duckFactory.createDuckCall();
Quackable rubberDuck = duckFactory.createRubberDuck();
Quackable gooseDuck = new GooseAdapter(new Goose());

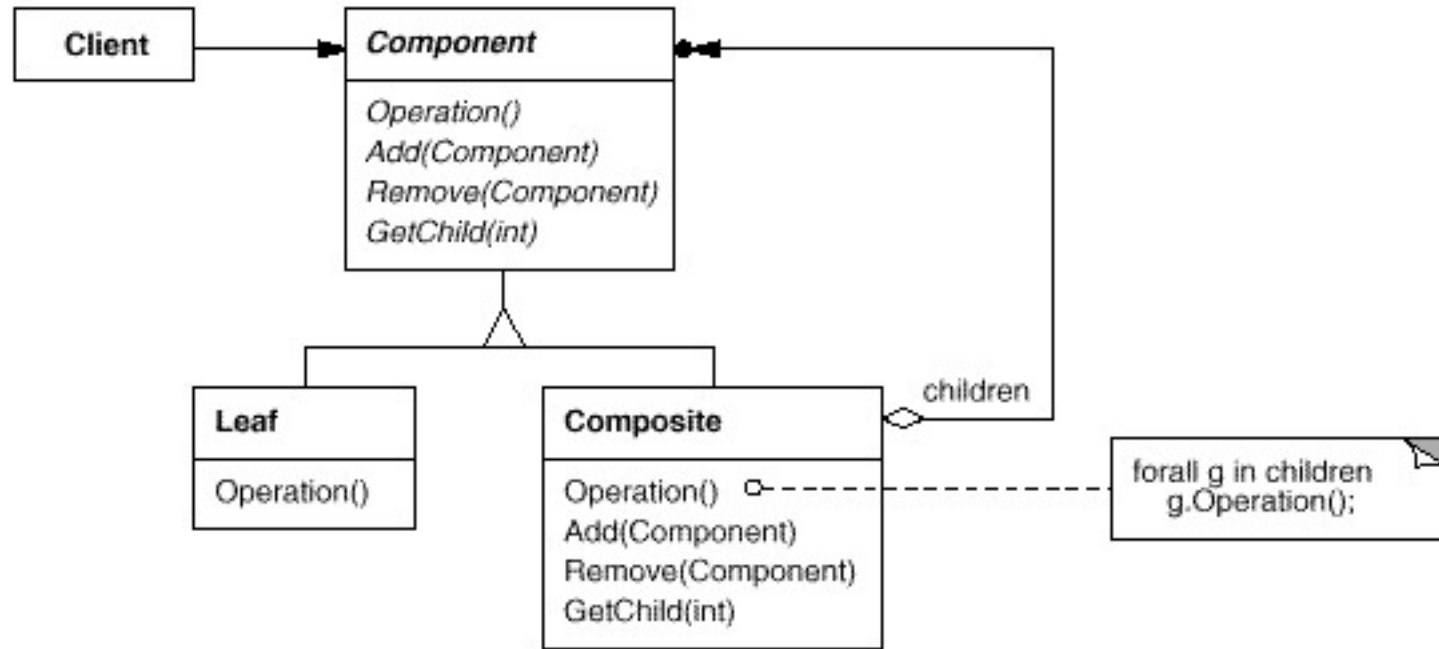
simulate(mallardDuck);
simulate(redheadDuck);
simulate(duckCall);
simulate(rubberDuck);
simulate(gooseDuck);
```


Manage a flock of ducks

- What we need is a way to talk about collections of ducks and even sub-collections of ducks (to deal with the family request from Ranger Brewer). It would also be nice if we could apply operations across the whole set of ducks.
- What pattern can help us?

a flock of ducks (a flock of Quackables)

Remember the Composite Pattern that allows us to treat a collection of objects in the same way as individual objects? What better composite than a flock of Quackables!



a flock of ducks (a flock of Quackables)

Remember the Composite Pattern that allows us to treat a collection of objects in the same way as individual objects? What better composite than a flock of Quackables!

```
public class Flock implements Quackable {
    ArrayList quackers = new ArrayList();

    public void add(Quackable quacker) {
        quackers.add(quacker);
    }

    public void quack() {
        Iterator iterator = quackers.iterator();
        while (iterator.hasNext()) {
            Quackable quacker = (Quackable)iterator.next();
            quacker.quack();
        }
    }
}
```

Remember, the composite needs to implement the same interface as the leaf elements. Our leaf elements are Quackables.

We're using an ArrayList inside each Flock to hold the Quackables that belong to the Flock.

The add() method adds a Quackable to the Flock.

There it is! The Iterator Pattern at work!

Now for the quack() method – after all, the Flock is a Quackable too. The quack() method in Flock needs to work over the entire Flock. Here we iterate through the ArrayList and call quack() on each element.

Safety versus Transparency

- You might remember that in the Composite Pattern the composites and the leaf nodes had the *same* exact set of methods, including the add() method. Because they had the same set of methods, we could call methods on the leaf nodes that didn't really make sense. The benefit of this was that the distinction between leaves and composites was *transparent*: the client didn't have to know whether it was dealing with a leaf or a composite; it just called the same methods on both.
- Here, we've decided to keep the composite's child maintenance methods separate from the leaf nodes: that is, only Flocks have the add() method. We know it doesn't make sense to try to add something to a Duck, and in this implementation, you can't. You can only add() to a Flock. So this design is *safer* – you can't call methods that don't make sense on components – but it's less transparent. Now the client has to know that a Quackable is a Flock in order to add Quackables to it.
- As always, there are trade-offs when you do OO design and you need to consider them as you create your own composites.

Composite is ready

```
public class DuckSimulator {  
    // main method here  
  
    void simulate(AbstractDuckFactory duckFactory) {  
        Quackable redheadDuck = duckFactory.createRedheadDuck();  
        Quackable duckCall = duckFactory.createDuckCall();  
        Quackable rubberDuck = duckFactory.createRubberDuck();  
        Quackable gooseDuck = new GooseAdapter(new Goose());  
        System.out.println("\nDuck Simulator: With Composite - Flocks");  
    }  
}
```

Create all the
Quackables, just
like before.

```
Flock flockOfDucks = new Flock();
```

```
flockOfDucks.add(redheadDuck);  
flockOfDucks.add(duckCall);  
flockOfDucks.add(rubberDuck);  
flockOfDucks.add(gooseDuck);
```

```
Flock flockOfMallards = new Flock();
```

First we create a Flock, and
load it up with Quackables.

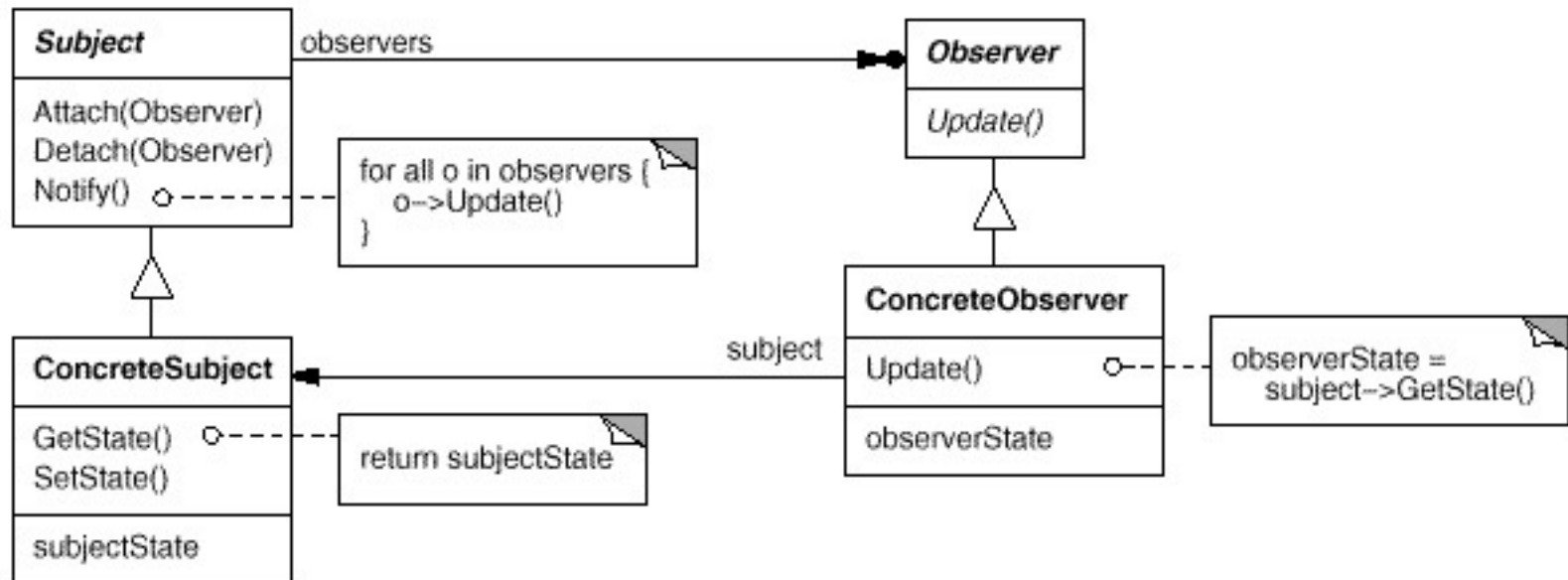
Then we create a new
Flock of Mallards.

Also track individual ducks

- The Composite is working great! Thanks! Now we have the opposite request: we also need to track individual ducks. Can you give us a way to keep track of individual duck quacking in real time?

Also track individual ducks

- The Composite is working great! Thanks! Now we have the opposite request: we also need to track individual ducks. Can you give us a way to keep track of individual duck quacking in real time?



Observer

First we need an **Observable** interface.

Remember that an Observable is the object being observed. An Observable needs methods for registering and notifying observers. We could also have a method for removing observers, but we'll keep the implementation simple here and leave that out.

```
public interface QuackObservable {  
    public void registerObserver(Observer observer);  
    public void notifyObservers();  
}
```

QuackObservable is the interface that Quackables should implement if they want to be observed.

It also has a method for notifying the observers.

It has a method for registering Observers. Any object implementing the Observer interface can listen to quacks. We'll define the Observer interface in a sec.

Now we need to make sure all Quackables implement this interface...

```
public interface Quackable extends QuackObservable {  
    public void quack();  
}
```

So, we extend the Quackable interface with QuackObserver.

QuackObservable

Now, we need to make sure all the concrete classes that implement Quackable can handle being a QuackObservable.

We could approach this by implementing registration and notification in each and every class. But we're going to do it a little differently this time: we're going to encapsulate the registration and notification code in another class, call it Observable, and compose it with a QuackObservable. That way we only write the real code once and the QuackObservable just needs enough code to delegate to the helper class Observable.

Observable implements all the functionality a Quackable needs to be an observable. We just need to plug it into a class and have that class delegate to Observable.

Observable must implement QuackObservable because these are the same method calls that are going to be delegated to it.

```
public class Observable implements QuackObservable {  
    ArrayList observers = new ArrayList();  
    QuackObservable duck;  
  
    public Observable(QuackObservable duck) {  
        this.duck = duck;  
    }  
  
    public void registerObserver(Observer observer) {  
        observers.add(observer);  
    }  
  
    public void notifyObservers() {  
        Iterator iterator = observers.iterator();  
        while (iterator.hasNext()) {  
            Observer observer = (Observer) iterator.next();  
            observer.update(duck);  
        }  
    }  
}
```

In the constructor we get passed the QuackObservable that is using this object to manage its observable behavior. Check out the notify() method below; you'll see that when a notify occurs, Observable passes this object along so that the observer knows which object is quacking.

Here's the code for registering an observer.

And the code for doing the notifications.

Integrate the helper Observable with the Quackable classes

All we need to do is make sure the Quackable classes are composed with an Observable and that they know how to delegate to it.

```
public class MallardDuck implements Quackable {  
    Observable observable;
```

Each Quackable has an Observable instance variable.

```
    public MallardDuck() {  
        observable = new Observable(this);  
    }
```

In the constructor, we create an Observable and pass it a reference to the MallardDuck object.

```
    public void quack() {  
        System.out.println("Quack");  
        notifyObservers();  
    }
```

When we quack, we need to let the observers know about it.

```
    public void registerObserver(Observer observer) {  
        observable.registerObserver(observer);  
    }
```

```
    public void notifyObservers() {  
        observable.notifyObservers();  
    }
```

Here's our two QuackObservable methods. Notice that we just delegate to the helper.

```
}
```

Observer

We've implemented everything we need for the Observables; now we need some Observers. We'll start with the Observer interface:

```
public interface Observer {  
    public void update(QuackObservable duck);  
}
```

Observer

We need to implement the *Observable* interface or else we won't be able to register with a *QuackObservable*.



```
public class Quackologist implements Observer {  
  
    public void update(QuackObservable duck) {  
        System.out.println("Quackologist: " + duck + " just quacked.");  
    }  
}
```



The *Quackologist* is simple; it just has one method, *update()*, which prints out the *Quackable* that just quacked.

Simulator

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        AbstractDuckFactory duckFactory = new CountingDuckFactory();

        simulator.simulate(duckFactory);
    }

    void simulate(AbstractDuckFactory duckFactory) {

        // create duck factories and ducks here

        // create flocks here

        System.out.println("\nDuck Simulator: With Observer");
        Quackologist quackologist = new Quackologist();
        flockOfDucks.registerObserver(quackologist);

        simulate(flockOfDucks);

        System.out.println("\nThe ducks quacked " +
                           QuackCounter.getQuacks() +
                           " times");
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

← All we do here is create a Quackologist and set him as an observer of the flock.

← This time we'll we just simulate the entire flock.

Let's give it a try and see how it works!

